

Práctico 2

Compilación y linking

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento presenta la resolución integral del Práctico 2 de la materia Sistemas Operativos (UNRC, 2026), centrado en el *toolchain* de C: las etapas de pre-procesamiento, compilación, ensamblado y linking, junto con la diferencia entre bibliotecas estáticas y dinámicas. Cada ejercicio se justifica con la teoría del capítulo *Herramientas de desarrollo* de las *Notas del curso* (Marcelo Arroyo) y se acompaña con comandos, salidas y fragmentos de código verificados sobre macOS arm64 (formato Mach-O). El razonamiento es portable: en sistemas GNU/Linux las mismas ideas se manifiestan sobre el formato ELF y herramientas como `objdump`, `readelf` y `nm`.

Índice

1. Introducción	1
2. Ejercicio 1 — Pre-procesado, assembly y objeto de <code>hello.c</code>	2
3. Ejercicio 2 — Compilar <code>main.c</code> a <code>main.o</code>	3
4. Ejercicio 3 — Análisis de <code>main.o</code> y <code>hello.o</code>	4
5. Ejercicio 4 — Generar el ejecutable <code>myprog</code>	6
6. Ejercicio 5 — Análisis assembly de <code>myprog</code>	7
7. Ejercicio 6 — Direcciones de variables locales (<code>stack</code>)	8
8. Ejercicio 7 — Biblioteca estática <code>libhello.a</code>	10
9. Ejercicio 8 — Biblioteca dinámica y <code>dlopen</code>	11
10. Ejercicio 9 — Automatización con <code>Makefile</code>	13
Cierre	15
Referencias	16

1. Introducción

Programar en C parece resumirse a un único comando —`gcc -o myprog main.c hello.c`—, pero detrás se esconde una secuencia de herramientas conocida como *toolchain*. Las Notas del curso la descomponen en cuatro etapas:

1. **Pre-procesamiento (cpp).** Expande las directivas `#include`, `#define` y demás macros. Su salida es un archivo `.i` con C ya expandido.

2. **Compilación (cc1)**. Traduce el C a *assembly* de la arquitectura objetivo. Salida: `.s`.
3. **Ensamblado (as)**. Convierte el assembly en un archivo objeto binario (`.o`) en el formato del SO (ELF en GNU/Linux, Mach-O en macOS).
4. **Linking (ld)**. Combina archivos objeto y bibliotecas, recalcula direcciones y resuelve referencias externas para producir el *ejecutable* final.

Las opciones `-E`, `-S` y `-c` de `gcc` permiten detener el proceso después de la primera, segunda y tercera etapa respectivamente.

Cada archivo objeto está estructurado en *secciones*: `.text` para el código, `.data` para variables globales con inicialización, `.rodata` (o `__cstring`) para constantes, una *tabla de símbolos* y una *tabla de reubicación* que indica qué instrucciones tienen operandos pendientes de resolver. El linker realiza dos tareas conceptuales: (1) concatenar las secciones de todos los objetos y recalcular direcciones, y (2) resolver las referencias externas usando las tablas mencionadas. Las llamadas a funciones definidas en el mismo ejecutable se resuelven en *linking estático*; las que viven en una biblioteca dinámica (como `printf` en `libc`) se resuelven en *linking dinámico* mediante *stubs* y la *Global Offset Table* (GOT).

Sobre estas bases, el práctico ejercita progresivamente los conceptos: generar artefactos intermedios (`.i`, `.s`, `.o`), inspeccionarlos con `objdump/otool`, observar cómo el linker resuelve direcciones, comprobar que las variables locales viven en la pila, y empaquetar código en bibliotecas estáticas y dinámicas —incluyendo carga explícita en tiempo de ejecución con `dlopen()`.

2. Ejercicio 1 — Pre-procesado, assembly y objeto de `hello.c`

Programa fuente

```
1  /* hello.c */
2  #define hi "Hello world"
3
4  char *hello(void)
5  {
6      return hi;
7  }
```

Notar que `hi` no es una variable: es una **macro**. El preprocesador la reemplaza textualmente por la cadena `"Hello world"` antes de que el compilador vea el código.

1.a Pre-procesamiento (`hello.i`)

```
gcc -E hello.c -o hello.i
```

El contenido relevante de `hello.i` es:

```
1  # 1 "hello.c"
2  # 1 "<built-in>" 1
3  ...
4  char *hello(void)
5  {
6      return "Hello world";
7  }
```

`hi` ya no aparece: la macro fue expandida y reemplazada por el literal `"Hello world"`. Las líneas `# n "archivo"` son marcadores que el preprocesador deja para que los mensajes de error

apunten al archivo y línea originales.

1.b Assembly (hello.s)

```
gcc -S hello.c -o hello.s
```

```
.section __TEXT,__text,regular,pure_instructions
.globl _hello
.p2align 2
_hello:
    adrp x0, l_.str@PAGE
    add x0, x0, l_.str@PAGEOFF
    ret
.section __TEXT,__cstring,cstring_literals
l_.str:
    .asciz "Hello world"
```

- `__TEXT,__text` contiene el código de `hello`.
- `__TEXT,__cstring` contiene el literal, marcado con la etiqueta local `l_.str`.
- ARM64 codifica direcciones en páginas de 4 KiB, por eso la dirección del literal se arma en dos pasos: `adrp` carga la base de la página y `add` suma el offset dentro de ella.
- El valor de retorno se deposita en `x0` (convención AAPCS64) y la función vuelve con `ret`.

1.c Archivo objeto (hello.o)

```
gcc -c hello.c -o hello.o
file hello.o # -> Mach-O 64-bit object arm64
```

`hello.o` es un binario relocatable Mach-O. Ya contiene el código máquina de `hello()`, pero su dirección sigue siendo `0x0`: la asigna el linker más adelante, al combinarlo con otros objetos.

3. Ejercicio 2 — Compilar `main.c` a `main.o`

```
1  /* main.c */
2  #include <stdio.h> /* prototipo de printf */
3
4  extern char* hello(void); /* hello vive en otro modulo */
5
6  int main(void)
7  {
8      printf("%s\n", hello());
9      return 0;
10 }
```

Tres elementos relevantes:

- `#include <stdio.h>` introduce el *prototipo* de `printf` para que el compilador valide los tipos de argumentos. La definición vive en la `libc`.
- `extern char* hello(void)` es una *declaración*; no necesita el cuerpo, solo el perfil.
- `printf` y `hello` aparecerán en `main.o` como símbolos externos no resueltos (`*UND*`).

```
gcc -c main.c -o main.o
file main.o # -> Mach-O 64-bit object arm64
```

La opción `-c` detiene el proceso tras el ensamblado: preprocesa, compila y ensambla, pero no linkea. `main.o` contiene el código de `main` y un literal `"%s\n"`, pero las llamadas a `hello` y `printf` todavía no están resueltas.

Compilar a objetos en lugar de directamente al ejecutable habilita la *compilación incremental*: si más adelante se modifica `hello.c` basta con regenerar `hello.o` y volver a linkear, sin recompilar `main.c`. Esa propiedad es la que explota `make`.

4. Ejercicio 3 — Análisis de `main.o` y `hello.o`

3.a Tipo de archivo objeto

```
file main.o hello.o
```

```
main.o: Mach-O 64-bit object arm64
hello.o: Mach-O 64-bit object arm64
```

Ambos son objetos relocatables Mach-O para arm64, equivalentes a un `.o` ELF en Linux. Las direcciones todavía no están resueltas.

3.b Secciones (`objdump -h`) y tamaños

```
main.o:
 0 __text 00000040 0000000000000000 TEXT
 1 __cstring 00000004 0000000000000040 DATA
 2 __compact_unwind 00000020 0000000000000048 DATA

hello.o:
 0 __text 0000000c 0000000000000000 TEXT
 1 __cstring 0000000c 000000000000000c DATA
 2 __compact_unwind 00000020 0000000000000018 DATA
```

Objeto	.text	.data	__cstring
main.o	0x40 (64 B)	— (sección ausente)	0x04: "%s\n"
hello.o	0x0c (12 B)	— (sección ausente)	0x0c: "Hello world"

Cuadro 1: Tamaños de secciones por objeto.

`main.o` es más grande porque tiene prólogo/epílogo de `main`, armado de argumentos de `printf` y dos llamadas (`hello`, `printf`). `hello()` solo arma un puntero y retorna.

No hay sección `__DATA`, `__data` en ningún objeto: el programa **no tiene variables globales con valor inicial distinto de cero**. El literal `"Hello world"` no va en `.data` sino en `__cstring` porque es de solo lectura.

3.c Contenido de cada sección

```
main.o (objdump -s):
Contents of section __TEXT,__text:
```

```

0000 ff8300d1 fd7b01a9 fd430091 08008052
0010 e80b00b9 bfc31fb8 00000094 e8030091
0020 000100f9 00000090 00000091 00000094
0030 e00b40b9 fd7b41a9 ff830091 c0035fd6
Contents of section __TEXT,__cstring:
0040 25730a00 %s..

hello.o (objdump -s):
Contents of section __TEXT,__text:
0000 00000090 00000091 c0035fd6 ....._
Contents of section __TEXT,__cstring:
000c 48656c6c 6f20776f 726c6400 Hello world.

```

Los bytes 25 73 0a 00 son la secuencia ASCII de "%s\n\n0"; los de `hello.o` forman "Hello world\n0".

3.d Direcciones de `main()` y `hello()`

```

main.o:
0000000000000000 g F __TEXT,__text _main

hello.o:
0000000000000000 g F __TEXT,__text _hello

```

Ambas funciones empiezan en el offset 0x0 de su sección `__text`. Esto es lo *normal* en archivos objeto relocatables: cada módulo se siente solo en el mundo. El linker luego apila las secciones y reasigna direcciones absolutas.

3.e Dónde se almacena el literal apuntado por `hi`

`hi` es la macro `#define hi "Hello world"`: *no* es una variable, así que no hay símbolo `hi` en la tabla de símbolos. Lo que sí hay es el literal al que la macro se expande, guardado en `hello.o`:

```

0000000000000000c 1 0 __TEXT,__cstring l_.str

```

Es decir, el string vive en la sección `__cstring` (de solo lectura, equivalente a `.rodata` en ELF), con etiqueta interna `l_.str`, en el offset 0x0c de `hello.o`. La función `hello()` arma esa dirección con `adrp+add` y la devuelve en `x0`.

3.f Tabla de símbolos y de reubicación

`main.o` (símbolos):

```

0000000000000000 1 F __TEXT,__text ltmp0
0000000000000040 1 0 __TEXT,__cstring l_.str
0000000000000040 1 0 __TEXT,__cstring ltmp1
0000000000000048 1 0 __LD,__compact_unwind ltmp2
0000000000000000 g F __TEXT,__text _main
0000000000000000 *UND* _hello
0000000000000000 *UND* _printf

```

`_main` está definido (g global, en `__text`). `_hello` y `_printf` aparecen como `*UND*`, indicando que son referencias externas. El linker las resolverá: `_hello` contra `hello.o` (linking estático), `_printf` contra `libSystem.B.dylib` (linking dinámico).

`main.o` (reubicaciones):

```
RELOCATION RECORDS FOR [__text]:
000000000000002c ARM64_RELOC_BRANCH26 _printf
0000000000000028 ARM64_RELOC_PAGEOFF12 l_.str
0000000000000024 ARM64_RELOC_PAGE21 l_.str
0000000000000018 ARM64_RELOC_BRANCH26 _hello
```

Esto es exactamente el mapping $relloc(instr_address, instr_type) \rightarrow external_symbol$ descrito en las notas: en el offset 0x18 hay un bl cuyo destino debe completarse con `_hello`; en 0x24/0x28 un par adrp/add que arma la dirección del literal; en 0x2c otro bl cuyo destino es `_printf`.

`hello.o` solo tiene un símbolo global definido (`_hello`) y dos entradas de reubicación para armar la dirección del literal local `l_.str`. No tiene símbolos `*UND*` porque `hello()` no llama a nadie.

3.g Instrucciones call en main()

```
0000000000000000 <_main>:
  0: sub sp, sp, #0x20
  4: stp x29, x30, [sp, #0x10]
  ...
18: bl 0x18 <tmp0+0x18> ; <-- call a _hello (sin resolver)
1c: mov x8, sp
20: str x0, [x8]
24: adrp x0, 0x0 <tmp0>
28: add x0, x0, #0x0
2c: bl 0x2c <tmp0+0x2c> ; <-- call a _printf (sin resolver)
...
3c: ret
```

Los dos bl apuntan a *sí mismos*: sus operandos están en 0. Esto es exactamente lo que las notas del curso describen: en el archivo objeto los call a símbolos externos quedan sin resolver y aparecen en la tabla de reubicación. **Los operandos no están resueltos**; los completará el linker.

3.h Cómo retornan los valores las funciones

Por la convención AAPCS64 de ARM64:

- El valor de retorno entero/puntero va en x0 (o w0 para 32 bits).
- La instrucción `ret` salta a la dirección guardada en x30 (*Link Register*).

`hello()` arma la dirección de la cadena en x0 y hace `ret`. `main()` guarda el puntero recibido como argumento para `printf` y devuelve el `int 0` en w0 antes del `ret`. En x86-64 la idea sería la misma, pero el registro de retorno sería `rax`.

5. Ejercicio 4 — Generar el ejecutable myprog

```
gcc main.o hello.o -o myprog
./myprog
# Hello world
```

¿Se recompilaron los programas? No. Cuando los argumentos de `gcc` son archivos objeto, `gcc` actúa solo como *driver* del linker. La verificación con `-v`:

```
gcc -v main.o hello.o -o myprog_from_objects
```

muestra exclusivamente la invocación a ld:

```
Apple clang version 17.0.0 ...  
".../usr/bin/ld" ... -o myprog_from_objects ... main.o hello.o -lSystem ...
```

No aparecen pasos de `cc1` o `as`: el código C no se toca. Si los argumentos hubieran sido `main.c` y `hello.c`, la traza mostraría las cuatro etapas (preprocesador, compilador, ensamblador y linker) corriendo para cada `.c`.

Qué hace el linker. Las notas describen el linking en dos pasos: (1) concatenar las secciones `.text/.data` de los objetos y *recalcular* direcciones; (2) resolver las referencias externas usando las tablas de símbolos y reubicación. En `myprog`:

- `_main` (de `main.o`) queda en `0x100000460`.
- `_hello` (de `hello.o`) queda en `0x1000004a0`, inmediatamente después.
- El `bl _hello` de `main` se completa con la dirección real (linking estático).
- El `bl _printf` se enlaza a un *symbol stub* del propio ejecutable que se resolverá dinámicamente.

6. Ejercicio 5 — Análisis assembly de myprog

5.a Direcciones de `main()` y `hello()`

```
$ nm -n myprog  
                U _printf  
0000000100000000 T __mh_execute_header  
0000000100000460 T _main  
00000001000004a0 T _hello
```

Ambos símbolos están definidos en el segmento `__TEXT`, uno detrás del otro. Es la primera tarea del linker: tomó las secciones `__text` de `main.o` y `hello.o` (que empezaban en offset 0) y las concatenó asignándoles direcciones absolutas únicas. `_printf` aparece como `U` (*undefined*) porque su dirección la resuelve `dyld` en ejecución.

5.b ¿`main -> hello` fue resuelto por el linker?

Sí, en linking estático.

```
_main:  
0000000100000460 sub sp, sp, #0x20  
...  
0000000100000478 bl _hello ; <-- llamada directa  
000000010000047c mov x8, sp  
0000000100000480 str x0, [x8]  
0000000100000484 adrp x0, 0 ; 0x100000000  
0000000100000488 add x0, x0, #0x4b8 ; "%s\n"  
000000010000048c bl 0x1000004ac ; symbol stub for: _printf  
000000010000049c ret  
_hello:  
00000001000004a0 adrp x0, 0 ; 0x100000000  
00000001000004a4 add x0, x0, #0x4bc ; "Hello world"
```

```
000000001000004a8 ret
```

El `b1` en `0x...478` ahora codifica el desplazamiento real al inicio de `_hello`. La invocación quedó *resuelta* porque ambos símbolos están definidos en el mismo ejecutable.

5.c ¿Cómo se resolvió la invocación a `printf()`?

`printf` vive en `libSystem.B.dylib` (la `libc` de macOS), una biblioteca *dinámica*. El linker estático no puede poner una dirección absoluta porque el cargador puede ubicar la `libc` en direcciones distintas en cada ejecución (ASLR). La técnica usada es la del **symbol stub** con *lazy linking* descrita en las notas:

1. `ld` genera un stub `_printf$stub` en `__TEXT,__stubs`.
2. `main` hace `b1 0x1000004ac` a ese stub.
3. El stub realiza un `jmp` indirecto a través de la *Global Offset Table* (`__DATA_CONST,__got`).
4. En la primera invocación, la GOT apunta al *dynamic linker* (`dyld`), que resuelve la dirección real de `printf` en `libSystem.B.dylib` y la escribe en la GOT.
5. Las invocaciones siguientes ya encuentran la dirección verdadera y saltan directamente.

Verificación:

```
$ otool -Iv myprog
Indirect symbols for (__TEXT,__stubs) 1 entries
address index name
0x000000001000004ac 3 _printf
Indirect symbols for (__DATA_CONST,__got) 1 entries
address index name
0x00000000100004000 3 _printf

$ otool -L myprog
myprog:
    /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, ...)
```

Llamada	Resuelve	Cuándo
<code>main -> hello</code>	Linker estático (<code>ld</code>)	En tiempo de linking
<code>main -> printf</code>	Linker dinámico (<code>dyld</code>)	En tiempo de carga / 1. ^a llamada

Cuadro 2: Resolución estática vs. dinámica en `myprog`.

7. Ejercicio 6 — Direcciones de variables locales (stack)

Marco. El espacio de direcciones de un proceso se divide en regiones bien definidas: `.text` (código, solo lectura), datos globales (`.data/.bss`), *heap* (memoria dinámica) y *stack* (pila de llamadas). Cada invocación a una función crea un *stack frame* con sus parámetros, variables locales, dirección de retorno y registros guardados. El compilador asigna las locales como desplazamientos relativos al puntero de marco (`x29/fp` en `arm64`). La pila crece hacia direcciones bajas.

Para no romper el `main.c` usado en otros ejercicios, la modificación se hizo en una variante: `main_stack.c`.

Idea del experimento.

1. Imprimir las direcciones de varias variables locales declaradas en distintas funciones.
2. Obtener *base* y *tope* del stack del hilo principal con `pthread_get_stackaddr_np/`
`pthread_get_stacksize_np`.
3. Verificar que cada dirección local cae dentro de `[base, tope)`.

```
1  /* main_stack.c (extracto) */
2  static void print_inner_locals(uintptr_t stack_base, uintptr_t stack_top)
3  {
4      int inner_a = 10;
5      int inner_b = 20;
6      char inner_buf[16] = "inner";
7      report_local("&inner_a", &inner_a, stack_base, stack_top);
8      report_local("&inner_b", &inner_b, stack_base, stack_top);
9      report_local("inner_buf", inner_buf, stack_base, stack_top);
10 }
11
12 static void print_stack_layout(void)
13 {
14     int local_a = 1;
15     int local_b = 2;
16     double local_c = 3.14;
17     char local_d = 'x';
18     char buffer[32] = "stack-local";
19     pthread_t self = pthread_self();
20     void *stack_top = pthread_get_stackaddr_np(self);
21     size_t stack_size = pthread_get_stacksize_np(self);
22     void *stack_base = (void *)((uintptr_t)stack_top - stack_size);
23     /* imprimir base, tope, tamaño y direcciones de cada local */
24     print_inner_locals((uintptr_t)stack_base, (uintptr_t)stack_top);
25 }
26
27 int main(void) {
28     printf("%s\n", hello());
29     print_stack_layout();
30     return 0;
31 }
```

Compilación y ejecución:

```
gcc -Wall -Wextra -pedantic main_stack.c hello.c -o myprog_stack
./myprog_stack
```

Salida típica (las direcciones absolutas cambian entre ejecuciones por ASLR):

```
Hello world
Stack del hilo principal:
  base (dir baja): 0x16a778000
  tope (dir alta): 0x16af74000
  tamaño: 8372224 bytes (7.98 MiB)

Locales en main/print_stack_layout:
  &local_a 0x16af7261c [STACK]
  &local_b 0x16af72618 [STACK]
  &local_c 0x16af72610 [STACK]
  &local_d 0x16af7260f [STACK]
  buffer 0x16af72620 [STACK]

Locales en funcion secundaria:
```

```
&inner_a 0x16af7255c [STACK]
&inner_b 0x16af72558 [STACK]
inner_buf 0x16af72570 [STACK]
```

Análisis.

- Todas las direcciones impresas caen dentro del rango `[stack_base, stack_top)`: las variables locales viven en el **stack**.
- El stack del hilo principal mide ≈ 8 MiB (8372224 B), valor por defecto en macOS y en muchos Linux (`ulimit -s`).
- Las locales de `print_inner_locals` están en direcciones *más bajas* que las de `print_stack_layout`, evidenciando que cada llamada empuja un nuevo frame hacia direcciones decrecientes.
- El alineamiento se respeta: `local_c` (un `double`) cae en una dirección múltiplo de 8.

Respuesta. Las variables locales se asignan en el *stack* del proceso, una región del espacio de direcciones que crece hacia direcciones bajas y se organiza en *stack frames* (uno por invocación de función). ASLR aleatoriza las direcciones absolutas en cada ejecución, pero la estructura relativa (frames internos por debajo de los externos) permanece.

8. Ejercicio 7 — Biblioteca estática libhello.a

Marco. Las notas definen una **biblioteca** como una colección de módulos reusables. Una biblioteca *estática* es un *archiver* (formato `.a` en UNIX) con un índice de símbolos. El linker, ante una referencia sin resolver, copia del `.a` solo los `.o` que la satisfacen y los enlaza en el ejecutable.

Para no romper `main.c`, este ejercicio usa una variante (`main_lib.c`) y un segundo módulo (`hello2.c`):

```
1  /* hello2.c */
2  #define hi2 "Hello from hello2"
3  char *hello2(void) { return hi2; }
```

```
1  /* main_lib.c */
2  #include <stdio.h>
3  extern char *hello(void);
4  extern char *hello2(void);
5  int main(void) {
6      printf("%s\n", hello());
7      printf("%s\n", hello2());
8      return 0;
9  }
```

7.a Crear la biblioteca estática

```
gcc -Wall -Wextra -pedantic -c hello.c -o hello.o
gcc -Wall -Wextra -pedantic -c hello2.c -o hello2.o
ar rcs libhello.a hello.o hello2.o
```

Las opciones de `ar` significan: `r` reemplaza miembros existentes, `c` crea el archivo si no existe y `s` genera/actualiza el índice de símbolos. El índice es lo que permite al linker responder rápidamente "¿qué objeto define a `hello`?".

7.b Listar los módulos de la biblioteca

```
$ ar -t libhello.a
__SYMDEF SORTED
hello.o
hello2.o
```

La primera entrada (`__SYMDEF SORTED`) es el índice de símbolos interno; los módulos reales son `hello.o` y `hello2.o`.

7.c Compilar enlazando con la biblioteca

```
gcc -Wall -Wextra -pedantic main_lib.c -L. -lhello -o myprog_lib
./myprog_lib
```

Salida:

```
Hello world
Hello from hello2
```

Detalles del comando:

- `-L.` agrega el directorio actual al *path* de búsqueda del linker.
- `-lhello` indica buscar una biblioteca llamada `libhello`. El prefijo `lib` y el sufijo (`.a/.dylib/.so`) los pone el linker.
- Si convivieran versiones estática y dinámica, *el linker prefiere la dinámica* salvo indicación explícita. En este ejercicio solo existe la estática.

El linker, al ver que `main_lib.o` referencia `_hello` y `_hello2`, busca en el índice de `libhello.a`, encuentra que provienen de `hello.o` y `hello2.o`, extrae esos miembros y los integra en el ejecutable. El resultado es un binario *autocontenido*: una vez linkeado no necesita la biblioteca para correr.

9. Ejercicio 8 — Biblioteca dinámica y `dlopen`

Marco. Las notas describen las **bibliotecas dinámicas** (`.so/.dylib/.dll`) como archivos que el SO carga y enlaza bajo demanda. El ejecutable no incluye su código: si varios procesos las usan, una sola copia en memoria los sirve a todos. La API POSIX `<dlfcn.h>` (`dlopen`, `dlsym`, `dlclose`) permite cargar bibliotecas *explícitamente* en ejecución, sin declarar la dependencia en tiempo de compilación.

8.a Generar la biblioteca dinámica

```
gcc -Wall -Wextra -pedantic -fPIC -shared hello.c -o libhello.dylib
file libhello.dylib
# -> Mach-O 64-bit dynamically linked shared library arm64
```

- `-fPIC` (*Position Independent Code*) es obligatorio: la biblioteca puede cargarse en distintas direcciones en cada proceso (ASLR), por lo que el código no puede asumir una dirección de carga fija.
- `-shared` le indica a `gcc` que la salida es una biblioteca compartida, no un ejecutable.

8.b/8.c Modificar main para carga dinámica

```

1  /* main_dlopen.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <dlfcn.h>
5
6  typedef char *(*hello_fn_t)(void);
7
8  int main(void)
9  {
10     const char *lib_path = "./libhello.dylib";
11     void *handle = dlopen(lib_path, RTLD_NOW);
12     if (handle == NULL) {
13         fprintf(stderr, "dlopen: %s\n", dlerror());
14         return 1;
15     }
16     dlerror(); /* limpia el estado de error previo */
17
18     hello_fn_t hello_fn = NULL;
19     *(void **>(&hello_fn) = dlsym(handle, "hello");
20     const char *err = dlerror();
21     if (err != NULL) {
22         fprintf(stderr, "dlsym: %s\n", err);
23         dlclose(handle);
24         return 1;
25     }
26
27     printf("%s\n", hello_fn());
28
29     dlclose(handle);
30     return 0;
31 }

```

Decisiones del diseño:

- `RTLD_NOW` resuelve todos los símbolos al cargar. `RTLD_LAZY` los resolvería bajo demanda; para una biblioteca pequeña es indistinto.
- `*(void **>(&hello_fn) = dlsym(...)` es el modismo recomendado por POSIX para evitar el *warning* sobre la conversión de `void *` a puntero a función.
- Se llama a `dlerror()` antes y después de `dlsym` para no confundir un símbolo con valor legítimamente `NULL` con un símbolo no encontrado.

Compilación y ejecución (en macOS la libc provee `dlopen`; en Linux haría falta `-ldl`):

```

gcc -Wall -Wextra -pedantic main_dlopen.c -o myprog_dlopen
./myprog_dlopen
# Hello world

```

8.c El código de hello no está en el ejecutable

```
$ otool -L myprog_dlopen
myprog_dlopen:
    /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, ...)
```

La única dependencia es `libSystem.B.dylib` (que provee `printf`, `dlopen`, etc.). **No hay referencia a `libhello.dylib`** porque el ejecutable no fue enlazado contra ella; la cargará el propio programa con `dlopen`. Una búsqueda del símbolo `_hello` en `myprog_dlopen` no devuelve nada: el código vive en la biblioteca dinámica y se carga en ejecución.

8.d Sección de enlazado dinámico

En Linux se usaría `objdump -T myprog2` para ver `.dynsym`. En macOS el equivalente es `otool`:

```
$ otool -L libhello.dylib
libhello.dylib:
    libhello.dylib (compatibility version 0.0.0, current version 0.0.0)
    /usr/lib/libSystem.B.dylib ...
```

`libhello.dylib` exporta el símbolo `_hello`, que es exactamente lo que `dlsym` resuelve.

Conexión con la teoría

- **Linking dinámico implícito** (ejercicios 4-5): el linker estático genera *stubs* y entradas en la GOT, y el *dynamic linker* resuelve los símbolos en tiempo de carga.
- **Carga dinámica explícita** (este ejercicio): el propio programa decide qué biblioteca cargar y cuándo, mediante `dlopen/dlsym`. Es la base de los *plugins* y de cualquier mecanismo de extensibilidad en ejecución.

10. Ejercicio 9 — Automatización con Makefile

Marco. Las notas del curso (sección *Build systems*) muestran que en un proyecto real los pasos del *toolchain* no se invocan a mano: se automatizan con un *build system*. El más clásico en UNIX es **GNU make**, basado en un archivo **Makefile** que describe los *objetivos* (*targets*), sus *dependencias* y los *comandos* para construirlos. La forma básica de una regla es

```
target: dependencies
<TAB>command
```

make construye un grafo de dependencias y, comparando los *timestamps* de cada archivo, decide qué reglas disparar. Si el objetivo es más nuevo que todas sus dependencias, no se reconstruye: de ahí la **compilación incremental**. Las notas también introducen las **variables automáticas** (`$@` = nombre del target, `$^` = todas las dependencias, `$<` = primera dependencia) y las **reglas de patrón** (`%.o: %.c`) que evitan repetir la misma acción para cada archivo fuente.

Organización del proyecto. Los originales `main.c` y `hello.c` viven en el directorio del práctico. Para mantener la convención de los demás ejercicios, el **Makefile** se ubica dentro de `ejercicio-9/` y referencia los fuentes vía la variable especial `VPATH`.

9.1 Makefile usado

```
# VPATH le dice a make donde buscar los archivos fuente cuando no
# estan en el directorio actual.
```

```

VPATH = ..

CC = gcc
CFLAGS = -Wall -Wextra -pedantic

# Objetivo por defecto: myprog
myprog: main.o hello.o
    $(CC) $(CFLAGS) -o $@ $^

main.o: main.c
    $(CC) $(CFLAGS) -c $< -o $@

hello.o: hello.c
    $(CC) $(CFLAGS) -c $< -o $@

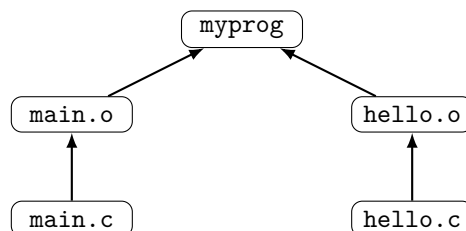
.PHONY: clean
clean:
    rm -f main.o hello.o myprog

```

Decisiones de diseño:

- `VPATH = ..` es la forma idiomática de separar fuentes y artefactos de compilación: `make` encuentra `main.c` y `hello.c` un nivel arriba y produce los `.o` y el ejecutable en el directorio actual.
- `CC` y `CFLAGS` centralizan las opciones del compilador en un solo lugar.
- Las variables automáticas `$@`, `$^` y `$<` hacen las reglas más concisas y reutilizables.
- `.PHONY: clean` declara `clean` como objetivo ficticio: si existiera un archivo llamado `clean`, `make` sin esto lo consideraría "ya construido" y no ejecutaría la regla.

9.2 Grafo de dependencias



`make myprog` requiere `main.o` y `hello.o`; `main.o` requiere `main.c`; `hello.o` requiere `hello.c`. `make` recorre el grafo en post-orden: primero compila los `.o` que falten o estén desactualizados y solo entonces invoca al linker.

9.3 Ejecución

Desde `ejercicio-9/`:

```

$ make clean
rm -f main.o hello.o myprog

$ make
gcc -Wall -Wextra -pedantic -c ../main.c -o main.o
gcc -Wall -Wextra -pedantic -c ../hello.c -o hello.o
gcc -Wall -Wextra -pedantic -o myprog main.o hello.o

$ ./myprog
Hello world

```

9.4 Compilación incremental

Si se vuelve a invocar `make` sin tocar nada, el ejecutable ya está al día y no se ejecuta ningún comando:

```
$ make
make: 'myprog' is up to date.
```

Si se modifica solo un archivo fuente, `make` rehace **únicamente lo necesario**:

```
$ touch ../hello.c
$ make
gcc -Wall -Wextra -pedantic -c ../hello.c -o hello.o
gcc -Wall -Wextra -pedantic -o myprog main.o hello.o
```

`main.o` no se recompiló porque `main.c` no cambió. Esta es la motivación central de `make`: en proyectos grandes, recompilar todo desde cero es inviable; `make` escala porque solo toca lo que tenga que tocar.

9.5 Versión compacta con regla de patrón

La misma funcionalidad puede escribirse aún más breve, tal como muestran las notas del curso, usando una regla de patrón `%.o : %.c` que define cómo construir cualquier `.o` a partir de un `.c`:

```
VPATH = ..
CC = gcc
CFLAGS = -Wall -Wextra -pedantic

myprog: main.o hello.o
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -f main.o hello.o myprog
```

La versión con reglas explícitas (la del Makefile activo) es más didáctica porque hace visibles cada uno de los tres comandos del toolchain. La versión con patrón es la que se usa en proyectos reales porque escala sin repetir reglas.

Conexión con la teoría. El `Makefile` no agrega capacidades nuevas al proceso de compilación; automatiza exactamente las mismas etapas vistas en los ejercicios 1–4: dos invocaciones a `gcc -c` (preprocesador, compilador y ensamblador) producen los `.o` y una invocación final al linker (también vía `gcc`) genera el ejecutable. Lo que `make` aporta es *automatización con conocimiento de dependencias*: la herramienta que vuelve manejables los proyectos grandes manteniendo compilaciones rápidas y reproducibles.

Cierre

A lo largo del práctico se "abrió" el proceso que normalmente queda oculto detrás del comando `gcc -o myprog ...`, y se observaron las cuatro etapas del *toolchain* en acción:

- Pre-procesamiento, compilación y ensamblado generan los artefactos intermedios (`.i`, `.s`, `.o`) cuyo contenido fue inspeccionado con `objdump` y `otool`.

- Los archivos objeto contienen secciones (`.text`, `.data`, `.rodata/__cstring`), una tabla de símbolos y una tabla de reubicación; en ellos las funciones empiezan en offset 0x0 y los `call` a símbolos externos quedan sin resolver.
- El linker concatena las secciones, recalcula direcciones y resuelve referencias: *estáticamente* las que se conocen (como `hello`) y *dinámicamente* las que viven en bibliotecas compartidas (como `printf`).
- Las variables locales se asignan en el *stack*, una región del espacio de direcciones del proceso que crece hacia direcciones bajas; se verificó experimentalmente que las direcciones impresas caen dentro del rango reportado por `pthread_get_stackaddr_np`.
- Las bibliotecas estáticas (`libhello.a`) son contenedores de `.o` con un índice de símbolos; el linker extrae solo los miembros necesarios y los integra al ejecutable.
- Las bibliotecas dinámicas (`libhello.dylib`) se cargan en tiempo de ejecución, ya sea de forma implícita (vía stubs y GOT) o explícita (vía `dlopen/dlsym`).
- Todo este flujo de compilación y enlace se automatiza con **make**: un **Makefile** declara objetivos, dependencias y comandos, y **make** reconstruye solo lo que cambió comparando *timestamps*.

Toda esta maquinaria es la base de los conceptos de los prácticos siguientes: cómo el SO carga un ejecutable, cómo organiza la memoria del proceso y cómo distintos procesos pueden compartir código mediante bibliotecas dinámicas.

Referencias

1. Marcelo Arroyo. *Notas del curso de Sistemas Operativos*, capítulo *Herramientas de desarrollo*. UNRC, 2026. <https://marceloarroyo.gitlab.io/cursos/SO/index.html#/toolchain>
2. Brian Kernighan, Dennis Ritchie. *The C programming language*, 2.^a edición. Prentice Hall.
3. John R. Levine. *Linkers and Loaders*. Morgan Kaufmann, 2000.
4. Páginas de manual: `man 1 gcc`, `man 1 ld`, `man 1 ar`, `man 1 objdump`, `man 1 otool`, `man 1 nm`, `man 3 dlopen`.